



CAPACITAÇÃO BACK-END

Roteiros do instrutor

Cronograma, o que falar em cada bloco, as demos ao vivo, as perguntas que a turma faz, e o plano B de cada encontro.

Fabício Siqueira

AUTOMIC JR. · 2026

Sumário

Guia do instrutor: conduzir, não apresentar

- Os três primeiros minutos
- "Não sei"
- A curva de energia de três horas
- Enquanto eles praticam
- Quando quebrar na sua frente
- Coisas que valem dizer em voz alta
- O que não fazer
- Sobre você
- O fecho do último dia

Roteiros: o índice para quem apresenta

- O número honesto
- A regra que organiza o dia
 - Como conduzir uma prática
 - As tabelas de erro
- As marcações nos slides
- Base do cálculo

Roteiro da Aula 1: back-end, Ruby e o primeiro Rails

- Antes de começar
- Cronograma
- Bloco a bloco
 - 00:00 · Abertura e o combinado (1–3)
 - 00:07 · Ferramentas e setup (4–11)
 - 00:30 · O que é back-end (12–16)
 - 00:42 · Rede (17–20)
 - 00:55 · HTTP e REST (21–30)
 - 01:23 · Prática 1: HTTP na mão (31)
 - 01:43 · Ruby (32–49)
 - 02:27 · Prática 2: Ruby no irb (50)
 - 02:37 · Rails (51–61)
 - 02:59 · Práticas 3 a 7: o projeto no ar (62, 65, 67)
 - 04:12 · Fecho (66, 68–69)

Perguntas que vão aparecer

Dever de casa

Roteiro da Aula 2: banco, ActiveRecord e o model User

Antes de começar

Cronograma

Bloco a bloco

00:00 · Retomada (1–3)

00:06 · Banco (4–9)

00:29 · ActiveRecord (11–16)

00:44 · Migrations (17–20)

01:29 · Senha (22–27)

01:57 · Validações (29–32)

02:46 · Associações (34–40)

03:21 · Concerns (42–45)

03:55 · Console e testes (47–51)

Conduzindo as oito práticas

Perguntas que vão aparecer

Dever de casa

Roteiro da Aula 3: as rotas de autenticação

A decisão que você precisa tomar antes

Opção A: código pronto, leitura guiada (recomendada)

Opção B: digitar junto

Antes de começar

Cronograma (Opção A)

Bloco a bloco

00:00 · Abertura e o combinado (1–5)

00:10 · Arquitetura (6–15)

00:56 · Cadastro (17–19)

01:36 · E-mail (21–24)

02:03 · JWT (26–33)

02:24 · Login (34–41)

03:05 · Logout (43–49)

03:42 · Recuperação de senha (51–56)

04:18 · Testes, ferramentas e CORS (58–62)

Conduzindo as oito práticas

Perguntas que vão aparecer

Dever de casa

Roteiro da Aula 4: servidor, Docker, Kamal e deploy

O que preparar com antecedência

O item 1 é o que tira o risco do dia

Antes de começar

Cronograma

Onde o tempo ainda pode escapar

Plano B, decidido de véspera

Bloco a bloco

00:06 · O problema e as opções (4–10)

00:24 · Chaves e SSH (11–18)

01:04 · Docker (20–25)

01:45 · Kamal (27–35)

02:22 · TLS (37–44)

03:13 · O deploy, e o certificado (51–52)

03:48 · Num servidor de verdade, demo (53–56)

04:23 · Prática 7 e fecho (57–62)

Perguntas que vão aparecer

Depois da capacitação

Guia do instrutor: conduzir, não apresentar

Os quatro roteiros dizem **o que** falar e **quando**. Este documento é sobre a outra metade: você, na frente de uma sala, por três horas, com gente que confia que você sabe.

Leia uma vez antes do primeiro encontro. Depois só volte no que precisar.

Os três primeiros minutos

São os mais difíceis do dia, e são os únicos que dá para ensaiar por completo. **Ensaie**. Em voz alta, de pé, umas três vezes.

O que precisa acontecer neles:

1. **Quem é você**, em duas frases. Não é currículo, é contexto: por que você está dando isso.
2. **De onde vem a capacitação**: é a continuação da do Fiuza. Lá foi o que o usuário vê.
3. **Onde eles chegam**: *"no fim do encontro 4, cada um vai ter uma API própria no ar, funcionando, que vocês construíram do zero."*

E então uma frase que muda o resto do dia:

"Vocês vão travar hoje. Todo mundo trava. Quando travar, levanta a mão — é para isso que eu estou aqui, e não para vocês fingirem que está tudo bem."

Dita no minuto três, ela dá permissão para a sala inteira. Sem ela, metade vai passar a aula escondendo que está perdida, e você só descobre na hora da prática — quando já custa caro.

"Não sei"

Vai acontecer. Alguém vai perguntar algo que você não sabe, e a sala vai estar olhando.

A resposta certa é a resposta curta:

"Não sei. Vou olhar e te respondo no grupo até amanhã."

E aí **anote na hora**, na frente deles, e responda mesmo. Isso não te diminui: mostra como um profissional lida com o limite do que sabe, que é uma das coisas mais úteis que eles vão levar do dia. Inventar uma resposta, por outro lado, é o único jeito de perder a sala de verdade — porque alguém vai conferir depois.

Se a pergunta for boa mas fora do escopo: "Ótima pergunta, e é assunto para uma conversa inteira. Me lembra no intervalo." E lembre você, se eles esquecerem.

A curva de energia de três horas

Ela é previsível, e dá para trabalhar com ela em vez de contra ela:

Quando	O que acontece	O que fazer
0:00–0:40	Atenção alta, ansiedade alta	Conteúdo denso aqui. É a sua melhor janela
0:40–1:20	Começa a cair	Primeira prática. Movimento salva
~1:20	Vale profundo	Intervalo. Não empurre.
1:30–2:20	Volta, mais baixa que no começo	Alterne conteúdo e prática a cada ~20 min
2:20–2:50	Cansaço real	Prática longa. Eles fazem, você circula
2:50–fim	Sobrevida curta	Fecho curto. Não comece assunto novo

Duas regras que saem daí:

- **Nunca dê mais de 20 minutos seguidos de slide.** Os decks são construídos para isso: cada bloco termina numa prática.
- **O intervalo não é negociável**, mesmo atrasado. Dez minutos comprados atropelando o intervalo custam quarenta de atenção depois.

Enquanto eles praticam

É o momento em que você mais ensina, e é fácil desperdiçá-lo.

Circule. Não fique no computador conferindo o próximo slide. Ande pela sala e olhe as telas. Quem está travado quase nunca chama; dá para ver pela postura antes de a pessoa falar.

Não digite no teclado dos outros. É rápido e é tentador, e rouba o aprendizado inteiro. Aponte a linha e diga o que está errado. Quem digita é a pessoa.

Quando três pessoas travarem no mesmo lugar, pare a sala. Não resolva individualmente pela quarta vez — projete, explique uma vez para todo mundo, e siga. Se três travaram, provavelmente outros seis estão travados e calados.

Quem terminou antes: toda prática tem um item marcado **avançado**. Use. Uma pessoa ociosa por 15 minutos desengaja e não volta.

Quem ficou muito para trás: pareie com quem terminou. Duas pessoas numa máquina que funciona aprendem; uma pessoa sozinha travada não aprende nada e ainda consome você. E deixe explícito que não é punição — é como se trabalha de verdade.

Quando quebrar na sua frente

Vai quebrar. Demo ao vivo quebra — é quase uma lei.

Não esconda e não acelere para disfarçar. **Leia o erro em voz alta e depure na frente deles.** Esse é, sem exagero, um dos momentos mais valiosos do dia: eles veem alguém que sabe lendo uma mensagem de erro com calma, em vez de entrar em pânico. É exatamente a habilidade que você está tentando ensinar.

Se em dois minutos não sair: *"vou deixar isso para depois e seguir"* — e siga de verdade. Insistir na frente da sala queima dez minutos e o clima.

Por isso os roteiros pedem que você faça o percurso inteiro na véspera, com o ambiente limpo. Não é burocracia: é para o erro que aparecer ser um que você já viu.

Coisas que valem dizer em voz alta

Guarde estas, e use quando couber. Cada uma resolve um problema real da sala:

"Isso aqui eu já perdi uma tarde inteira. Não é você, é a ferramenta mesmo."

Quando alguém trava num erro obscuro. Tira o peso na hora.

"Quem não está entendendo? Sério, levanta a mão. Eu prefiro voltar agora."

Perguntar "alguma dúvida?" quase nunca funciona — ninguém quer ser o único. Perguntar assim, esperando de verdade uns cinco segundos em silêncio, funciona.

"Não precisa decorar. Precisa saber que existe e onde procurar."

Toda vez que o assunto for lista de coisas (status codes, flags, comandos).

"Copiar e colar sem ler é o único jeito de sair daqui sem aprender nada."

No começo da primeira prática que tem bastante código.

"Errar aqui é o processo, não é falha."

Sempre que alguém pedir desculpa por ter travado. Eles pedem, e isso diz muito sobre como chegaram.

O que não fazer

- **Não leia os slides.** Eles são o apoio, você é a aula. Se você ler, a sala lê mais rápido que você fala e desengaja.
- **Não peça desculpa pelo material** ("desculpa, isso está meio corrido"). Reduz a confiança deles no que estão recebendo, e não conserta nada.
- **Não compare a turma com outra**, nem com você na idade deles.
- **Não responda três perguntas seguidas da mesma pessoa.** Redirecione: *"boa — alguém mais tem algo sobre isso?"*
- **Não termine com "acho que é isso".** Termine com a frase que você preparou.

Sobre você

Três horas falando cansa muito mais do que parece.

- **Água na mesa.** Não é conselho de palestrante, é logística: a voz vai embora.
- **Coma antes.** Não dá para pensar bem com fome no fim de um bloco de duas horas.
- **Não é o Oscar.** Se você entregar 70% do que planejou e a turma sair com a coisa funcionando, a aula foi ótima. O roteiro de tempo é explícito sobre isso: há mais conteúdo do que cabe, **de propósito**, para você escolher.
- **Vai dar errado alguma coisa.** Sempre dá. Você conduzir bem o que deu errado é o que a turma vai lembrar — mais do que o slide perfeito.

O fecho do último dia

Não termine no `curl` que funcionou. Reserve cinco minutos e diga, olhando para eles:

- **o que eles construíram**, listado: uma API com cinco fluxos de autenticação, testada, num servidor Linux que eles provisionaram, com HTTPS. Do zero, em quatro encontros. É muita coisa, e quem fez raramente percebe o tamanho.
- **o que vem depois:** o `seem-backend` está aberto, e agora eles conseguem ler.

- **que o projeto é deles:** está no GitHub deles, e serve de portfólio.

E deixe o canal aberto de verdade. A pergunta que chega três semanas depois costuma ser a mais importante da capacitação inteira.

Roteiros: o índice para quem apresenta

Um runbook por encontro, com cronograma, o que falar em cada bloco, as demos ao vivo, as perguntas que a turma faz e o plano B:

Antes de qualquer roteiro, leia o guia do instrutor. Ele não é sobre conteúdo: é sobre conduzir uma sala por três horas — os três primeiros minutos, o que fazer quando você não souber a resposta, onde a energia da turma cai, e o que dizer quando alguém travar.

	Roteiro	Slides	Práticas	Sem cortes	Com o plano de corte	Disponível
1	Back-end, Ruby e o primeiro Rails	69	7	4h17	3h21	3h
2	Banco, ActiveRecord e o model <code>User</code>	54	8	4h39	3h02	3h
3	As rotas de autenticação	66	8	5h06	3h19	3h
4	Servidor, Docker, Kamal e deploy	62	7	4h31	3h18	3h

O número honesto

São ~18h30 de conteúdo em 12h de encontro, e ~13h depois dos planos de corte. Cada roteiro traz os cortes específicos, numerados em ordem de prioridade, e um **ponto de decisão** com relógio: se às tantas horas a turma não estiver em tal lugar, faça tal coisa.

Não tente caber tudo. **Decida os cortes na véspera**, não no meio da aula.

E guarde o parágrafo que mais importa para você: **se a turma sair com a coisa funcionando, a aula foi ótima — mesmo que você tenha entregue 70% do que planejou**. Há mais conteúdo do que cabe, de propósito, para você escolher o que serve àquela turma.

A regra que organiza o dia

Quando atrasar, corte **conteúdo**, nunca **prática**. Uma turma que viu 40 slides e fez a coisa funcionar aprendeu mais que uma que viu 68 e foi embora com o erro na tela.

Os quatro decks são construídos em cima disso: **cada bloco de conteúdo termina numa prática**, e a prática está marcada em negrito no cronograma.

explica → faz → explica → faz → ...

Não é enfeite de estrutura. Antes, as aulas 2 e 3 tinham **uma** prática, no penúltimo slide: três horas ouvindo, e "agora façam tudo". Agora são 7 ou 8 por aula, e ninguém passa mais que ~25 min sem pôr a mão.

Como conduzir uma prática

1. **Projete o slide da prática** e leia os itens em voz alta. São 30 segundos.
2. **Mande abrir a apostila** na prática correspondente — os comandos, a conferência e a tabela de erros estão lá. Não dite comando.
3. **Circule**. As notas do slide dizem quais são os dois ou três erros que vão aparecer.
4. **Cobre a conferência** antes de seguir. Toda prática tem uma linha "Confere".

Cada prática tem, no fim, um item **avançado** para quem terminar antes. Use-o: é o que evita que metade da turma fique parada esperando a outra metade.

As tabelas de erro

Toda prática da apostila termina com uma tabela `Se der errado`, no formato **erro** → **causa** → **saída**. Elas cobrem os erros que acontecem de verdade, e existem para você não ser o único caminho de desbloqueio numa sala de 12 pessoas.

Quando alguém travar, a primeira pergunta é *"o que a tabela da sua prática diz?"*.

As marcações nos slides

Marca	Significa
# CORTÁVEL	Pré-requisito. Corte se a turma já sabe Git, terminal, SQL ou Linux.
# EXTRA	Conceito de fundo (rede, TLS, XSS...). O exercício funciona sem ele — mas é o que separa "copiei" de "entendi".

```
grep -n "CORTÁVEL\|EXTRA" slides/conteudo/aula-0*.yaml
```

Cortar todos os `# EXTRA` devolve ~45 slides, uns 90 minutos no total das quatro aulas. **A recomendação é o contrário**: mantenha os `# EXTRA`, e siga o plano de corte de cada roteiro.

Base do cálculo

Slide a ~2 min; digitação guiada a ~5 linhas/min; prática cronometrada no percurso real, incluindo o tempo de quem erra uma vez. Os tempos de download (`bundle install`, `apt install`, build da imagem) estão contados pelo pior caso de wi-fi compartilhado.

Roteiro da Aula 1: back-end, Ruby e o primeiro Rails

Deck: `slides/build/aula-01-fundamentos-de-back-end.pptx` (69 slides, sendo 5 de prática) **Apostila da turma:** `01-fundamentos.md` · **Checkpoint:** `aula-01` **Antes de tudo:** `guia-do-instrutor.md` — conduzir a sala, não o conteúdo

Antes de começar

Na semana anterior

- [] Mandar `00-preparacao.md` no grupo e cobrar resposta de quem travou.
- [] Levantar quem usa Windows → WSL2 instalado **antes**.
- [] Cobrar o teste do SSH (`ssh "$USER"@127.0.0.1 'echo ok'`): é da Aula 4, mas quem descobrir só no dia perde vinte minutos instalando `openssh-server` no meio da aula.

No dia, antes de a turma chegar

- [] Deck aberto, modo apresentador (as notas de cada slide estão preenchidas).
 - [] Dois terminais grandes: um para o `irb`, outro para o `rails`.
 - [] Fonte do terminal em pelo menos 18pt.
 - [] Insomnia aberto.
 - [] `curl -i https://api.seemxxiii.tech/api/v1/status` testado — é a sua demo do slide 27.
 - [] Gabarito em `~/capacidade-gabarito` na branch `aula-01`, pronto para projetar quem travar.
 - [] `gh auth status` funcionando no seu terminal — você vai demonstrar o `gh repo create`.
-

Cronograma

As sete práticas são o esqueleto do dia. Estão em negrito, e a regra é a de sempre: quando atrasar, corte **conteúdo**, nunca prática.

Relógio	Slides	Bloco	Min
00:00	1–3	Abertura, objetivos e o combinado de hoje	7
00:07	4–11	Ferramentas e setup	23
00:30	12–16	O que é back-end	12
00:42	17–20	Rede: servidor, IP, porta, DNS, URL	13
00:55	21–30	HTTP, JSON e REST	28
01:23	31	Prática 1 — HTTP na mão	10
01:33	—	Intervalo	10
01:43	32–49	Ruby para quem sabe Python	44
02:27	50	Prática 2 — Ruby no <code>irb</code>	10
02:37	51–61	Rails, MVC, Zeitwerk, ambientes	22
02:59	62	Práticas 3 e 4 — o projeto no ar	25
03:24	63–64	A primeira rota, e os dois diretórios	8
03:32	65	Prática 5 — a sua primeira rota	20
03:52	67	Práticas 6 e 7 — teste e commit	20
04:12	66, 68–69	Git, recapitulação, fim	5

Isso dá 4h17, e é a aula mais cheia das quatro. Não cabe em 3h sem você decidir antes o que sai. Corte **nesta ordem**, e decida na véspera:

#	O que cortar	Ganho
1	O bloco 4–11 inteiro (Ferramentas). O setup é <code>00-preparacao.md</code> , de casa — aqui você só roda a checagem final, em 3 min	–20
2	Slides 42–44 (argumentos nomeados, <code>Struct</code> , exceções). Aparecem na Aula 3, no código real	–10
3	Slides 52 (<code>O TERMINAL</code>) e 66 (<code>GIT</code>), os dois marcados <code># CORTÁVEL</code>	–7
4	Slides 55 (<code>DJANGO × RAILS</code>) e 60 (<code>TOUR DAS PASTAS</code>) — ficam na apostila	–6
5	Slide 20 (<code>DNS</code>) — volta na Aula 4 de qualquer jeito	–3
6	Práticas 6 e 7: faça só o teste em aula, e mande o commit de casa	–10

Cortando de 1 a 5, fecha em **3h31**. Cortando os seis, **3h21**.

O slide 46 (`case`) e a seta **não corte**: a seta aparece no model da Aula 2.

Bloco a bloco

00:00 · Abertura e o combinado (1-3)

Você se apresenta e diz de onde veio a capacitação: **é a continuação da do Fiuza**. Lá foi o que o usuário vê; aqui é o outro lado.

Nos objetivos, a frase que prende: *"no fim do encontro 4, cada um de vocês vai ter uma URL `https://` própria, funcionando, que qualquer um do mundo consegue chamar."*

E então o slide 3, que é o mais importante dos três primeiros minutos. Não passe por ele rápido. A frase precisa sair da sua boca, olhando para a sala:

"Vocês vão travar hoje. Todo mundo trava. Quando travar, levanta a mão — é para isso que eu estou aqui, e não para vocês fingirem que está tudo bem."

Dita agora, ela dá permissão para a sala inteira. Sem ela, metade vai passar a aula escondendo que está perdida, e você só descobre na hora da prática. Detalhes em `guia-do-instrutor.md`.

00:07 · Ferramentas e setup (4-11)

Passe rápido pelos slides e **pare no 10**. Todo mundo roda os quatro comandos ao mesmo tempo:

```
curl https://mise.run | sh
mise use --global ruby@3.4.9
gem install rails -v 8.1.3
docker run --rm hello-world
```

Peça para todos mostrarem a saída de `ruby -v` antes de seguir.

Ponto de não-retorno: se passou de 00:35 e ainda tem gente instalando, **pare**. Pareie quem travou com quem terminou e siga. Instalação é problema de casa, não de aula.

00:30 · O que é back-end (12-16)

O slide 12 é a ponte com a capacitação anterior: mostre a coluna do front e diga "isso vocês já viram". O slide 13 tem o ponto que importa: **tudo que roda no navegador o usuário consegue ler e alterar** — por isso a validação que vale é a do back.

O restaurante (14) funciona bem. Volte nele quando falar de API.

00:42 · Rede (17-20)

Aqui muita gente descobre coisa que usava sem saber.

- **16:** servidor não é máquina especial, é programa esperando numa porta.

- **17**: a tabela de portas. Diga que 22, 80 e 443 voltam na Aula 4, quando forem abrir e fechar firewall na mão.
- **19**: desmonte a URL na tela. Pergunte: "por que `localhost:3000` tem dois pontos e `google.com` não?" Deixe alguém responder.

00:55 • HTTP e REST (21–30)

Slide 22 é o coração do bloco. Uma requisição HTTP é texto puro. Se der, faça ao vivo:

```
curl -v https://api.seemxxiii.tech/api/v1/status
```

O `-v` mostra as linhas com `>` (o que foi) e `<` (o que voltou). É a mesma coisa do slide, acontecendo de verdade.

No **24** (`Content-Type`), avise: "esquecer esse cabeçalho é o erro nº 1 de vocês na Aula 3. Vem 400 e a mensagem não ajuda em nada."

No **26** (status codes), a pergunta: "qual a diferença entre 401 e 403?" — 401 é "quem é você?", 403 é "sei quem você é, e você não pode". Diga que cai em entrevista.

No **fim do bloco**, marque a frase: **HTTP não tem memória**. Escreva no quadro se tiver. É o problema inteiro da Aula 3.

01:23 • Prática 1: HTTP na mão (31)

A primeira vez que eles falam com um servidor sem navegador no meio. Dez minutos, e circule.

O erro de condução aqui é deixar rodarem os cinco comandos em sequência sem ler nada. Pare depois do `-v` e peça, em voz alta, que alguém aponte onde termina a requisição e onde começa a resposta.

A pergunta de fechamento, que vale a prática inteira:

"404 é erro de conexão?"

Não: é uma resposta perfeitamente bem-sucedida dizendo que aquele recurso não existe. Quem entende isso hoje não confunde 404 com 500 na Aula 3.

Quem terminar rápido: o item (e), o POST na mão com `-X`, `-H` e `-d`. É o que o Insomnia faz por baixo.

01:43 • Ruby (32–49)

O bloco mais longo. **Não leia os slides — rode no `irb` ao vivo.**

```
3.class          # Integer
"oi".class      # String
:oi.class       # Symbol
nil.class       # NilClass

3.times { puts "oi" }

usuario = { nome: "Ana", role: :admin }
usuario[:nome]

[1, 2, 3].map { |n| n * n }
[1, 2, 3].select { |n| n.odd? }
```

Pontos onde vale parar:

- **35** — as três diferenças: `end`, `if` no fim da linha, retorno implícito.
- **37** — símbolo não existe em Python. A regra: conteúdo que o usuário lê é string; nome de coisa no código é símbolo.
- **40** — `map` / `select` são iguais aos de Python. A diferença é que aqui são o caminho normal.
- **41** — o `?` e o `!`. Diga que o Rails inteiro depende dessa convenção.
- **42-43** — argumentos nomeados e `Struct`. Fale a frase: "o `user:` sozinho não é erro de digitação", porque eles vão ver isso em todo service da Aula 3.
- **46** — `case` e a seta `->`. A seta aparece no model da Aula 2; sem este slide ela lê como símbolo mágico.
- **47** — as três pegadinhas. Rode no `irb`: `7 / 2`, depois `7.0 / 2`. E `puts 'Olá #{1+1}'` com aspas simples. São 30 segundos e economizam um bug cada.

02:27 • Prática 2: Ruby no `irb` (50)

Dez minutos no `irb`, e é a **única vez** na capacitação em que eles mexem em Ruby sem Rails no caminho. Não corte, mesmo atrasado.

Os oito passos estão na apostila (Prática 2). Os dois que fixam, e que valem cobrar em voz alta:

- **por que** `usuario["nome"]` deu `nil`? — a chave é o símbolo `:nome`, não a string
- **por que** `saudacao("Ana")` deu `ArgumentError`? — o método pede argumento nomeado

E o passo (b), que é o que mais surpreende quem vem de Python:

```
puts "entrou" if 0      # entra! Só nil e false são falsos em Ruby.
```

Quem terminar rápido: o item (h), escrever um módulo e incluir numa classe — é a Aula 2 chegando mais cedo.

02:37 • Rails (51-61)

- **52** é a ideia central: você não configura o óbvio, você segue o combinado.
- **55** (Django × Rails) — pergunte quem já usou Django ou Flask e ancore neles.

- **58** (Zeitwerk) — a frase: *"não é estilo, é mecanismo. Errou o caminho, a classe não existe."*
- **63** — mostre a rota e o controller lado a lado, e aponte a convenção ligando os dois.
- **64** (`DOIS DIRETÓRIOS`) — **não corte**. É onde fica claro que o repositório da capacitação é gabarito e que o projeto é deles. Sem isso, metade da turma vai editar o repo errado.

02:59 · Práticas 3 a 7: o projeto no ar (62, 65, 67)

A entrega da aula, em três blocos separados por slides curtos. **Ninguém sai sem os 200 na tela e sem o projeto no GitHub deles.**

Prática 3 e 4 (slide 62, ~25 min) — criar o projeto, subir banco e aplicação:

```
rails new automic_auth_api --api -d postgresql \
  --skip-action-mailbox --skip-action-text --skip-active-storage \
  --skip-jbuilder --skip-action-cable
cd automic_auth_api
docker compose up -d          # tem que ficar healthy
bin/rails db:prepare
bin/rails server               # e abrir /up
```

É onde mais gente trava, e são sempre os mesmos dois:

Sintoma	Saída
<code>address already in use</code> na 5432	<code>DB_PORT=5433 docker compose up -d</code> , e <code>export DB_PORT=5433</code> no mesmo shell do <code>rails</code>
<code>permission denied</code> no <code>docker</code>	<code>sudo usermod -aG docker \$USER</code> , e sair e entrar de novo — reabrir a aba não basta

Ponto de decisão: se metade da turma não tiver o `/up` verde às 03:24, pare o conteúdo e resolva junto. Sem isso, as práticas 5 a 7 não acontecem.

Prática 5 (slide 65, ~20 min) — a rota. O erro clássico é `uninitialized constant Api::V1::StatusController`: o arquivo está no caminho errado. Volte ao slide 58 (Zeitwerk) e mostre a correspondência nome ↔ caminho na tela.

Práticas 6 e 7 (slide 67, ~20 min) — o teste e o commit. **Faça o passo de quebrar o teste de propósito** (`"ok"` → `"OK"`): são 30 segundos, e é o que muda a relação deles com teste.

Quem travar em qualquer uma: projete o arquivo do gabarito (`~/capacitacao-gabarito`, branch `aula-01`) e deixe a pessoa digitar. **Não mande copiar e colar** — o exercício inteiro são 20 linhas.

04:12 · Fecho (66, 68–69)

Recapitule em cinco frases (slide 68) e feche com o que vem: *"na próxima, a API ganha memória."*

Perguntas que vão aparecer

Pergunta	Resposta curta
"Por que Ruby e não Python?"	Porque o <code>seem-backend</code> é Ruby, e porque Rails entrega um sistema inteiro sem você montar peça por peça. E a linguagem é o de menos: o que vocês vão aprender aqui vale em qualquer uma.
"Rails não morreu?"	GitHub, Shopify e Basecamp rodam nele hoje. O que morreu foi o hype, não a ferramenta.
"Preciso decorar os status codes?"	Não. Precisa saber a faixa: 2xx deu certo, 4xx você errou, 5xx eu errei. O resto se consulta.
"Por que não usar o navegador para testar a API?"	O navegador só sabe fazer GET. Nossas rotas são POST e DELETE.
"Posso usar o Ruby que já está instalado no meu Linux?"	Pode dar certo e pode dar errado. É por isso que existe o mise: a versão exata, por projeto.
"O <code>--api</code> tira alguma coisa que vou precisar?"	Tira view, sessão de navegador e assets. Se um dia precisar, dá para trazer de volta — foi o que fizemos com os cookies.

Dever de casa

1. Terminar a prática, se não deu tempo.
2. Ler `ruby-para-pythonistas.md` inteiro.
3. Bônus da Prática 7: fazer a rota devolver `RUBY_VERSION` e `Rails.version`, com asserção no teste.

Roteiro da Aula 2: banco, ActiveRecord e o model `User`

Deck: `slides/build/aula-02-banco-de-dados-e-activerecord.pptx` (54 slides, sendo 8 de prática)

Apostila da turma: `02-activerecord.md` · **Checkpoint:** `aula-02` **Antes de tudo:** `guia-do-instrutor.md` — conduzir a sala, não o conteúdo

É a aula mais tranquila das quatro em termos de tempo. Use a folga para o console ao vivo — é o que mais fixa.

Antes de começar

- [] Gabarito em `~/capacitacao-gabarito`, branch `aula-02`, com o banco já preparado.
- [] `docker compose up -d` rodando no seu, para o console não travar na hora.
- [] Terminal grande, com o `bin/rails console` já aberto e testado.
- [] Ter em mãos um exemplo real de vazamento de senha para citar (LinkedIn 2012, 6,5 milhões de hashes SHA-1 sem salt — quebrados em dias).
- [] Conferir quem não terminou a prática da Aula 1 e resolver nos primeiros 10 minutos.

Cronograma

As oito práticas são o esqueleto do dia, uma por bloco: explica, faz, explica, faz. Estão em negrito, e quando atrasar você corta **conteúdo**, nunca prática.

Relógio	Slides	Bloco	Min
00:00	1–3	Abertura e retomada	6
00:06	4–9	Banco, transação, Postgres, container	18
00:24	10	Prática 1 — o banco de pé, e o bcrypt	5
00:29	11–20	ActiveRecord, <code>blank?</code> , migrations, <code>schema.rb</code>	30
00:59	21	Prática 2 — as três migrations	20
01:19	—	Intervalo	10
01:29	22–27	Senha: hash, bcrypt, <code>has_secure_password</code>	18
01:47	28	Prática 3 — o bcrypt no console	10
01:57	29–32	Validações e normalização	14
02:11	33	Práticas 4 e 5 — o validador e o model	35
02:46	34–40	Associações e N+1	20
03:06	41	Prática 6 — a segunda tabela	15
03:21	42–45	Concerns	14
03:35	46	Prática 7 — os dois concerns	20
03:55	47–51	Console ao vivo, seeds, fixtures, teste	15
04:10	52	Prática 8 — fixtures, testes e commit	25
04:35	53–54	Recapitulação e fim	4

Dá 4h39. É a aula em que eles mais escrevem código, e o tempo de digitação é real. Corte **nesta ordem**:

#	O que cortar	Ganho
1	Prática 8 vira dever de casa: em aula, só as fixtures e <code>bin/rails test</code> rodando	–15
2	Slide 34 (<code>O MODELO RELACIONAL</code> , <code># CORTÁVEL</code>) se a turma já viu SQL	–5
3	Prática 3 (bcrypt no console) vira demo sua, projetada, em 4 min	–6
4	Slide 40 (<code>A ARMADILHA N+1</code>) — fica na apostila e volta na Aula 3	–5
5	Práticas 4 e 5: dê o validador pronto (do gabarito) e eles só escrevem o <code>User</code>	–15
6	Prática 6: você faz projetado, eles copiam a migration	–8
7	Prática 7: dê o <code>EmailVerifiable</code> pronto e eles escrevem só o <code>PasswordResettable</code>	–8

Cortando de 1 a 4, fecha em **3h48**. Cortando os sete, **3h02**.

Não corte a Prática 2. Migration mal feita é o que trava a Aula 3 inteira, e o erro só aparece uma semana depois.

Bloco a bloco

00:00 · Retomada (1–3)

O slide 3 é a ponte: *"temos uma API que responde. Ela não guarda nada. Reiniciou, esqueceu tudo. Hoje ela ganha memória."*

00:06 · Banco (4–9)

Transação (6–7) é o slide que eles vão precisar na Aula 3. Use o exemplo do dinheiro: debitar de um e creditar no outro têm que ser a mesma operação; debitar sozinho é dinheiro que sumiu.

Aponte o `!` no slide 7 e explique: dentro da transação você **quer** que estoure, porque `save` devolve `false` em silêncio e a transação seguiria feliz gravando metade.

No 8, a frase: *"desenvolver no mesmo banco da produção elimina uma categoria inteira de bug."*

00:29 · ActiveRecord (11–16)

O slide 13 é o que causa espanto em quem vem de Django: **a classe não declara nada.**

```
class User < ApplicationRecord
end
```

Diga: *"isso já tem todas as colunas. A fonte da verdade é a migration, não a classe."*

15 é para rodar no console, não para ler:

```
nil.blank?      # true
"".blank?       # true
"  ".blank?     # true
0.blank?        # false    ← aqui alguém vai reclamar
```

E a pegadinha que vem de Python: em Ruby, `if 0` executa. `if ""` executa. Só `nil` e `false` são falsos.

00:44 · Migrations (17–20)

Gere uma migration ao vivo:

```
bin/rails generate migration CreateUsers
```

Mostre o arquivo criado, o timestamp no nome, e diga que é ele que define a ordem.

O slide 18 tem o ponto mais importante do bloco: índice único é restrição do banco, validação é mensagem bonita. Duas requisições simultâneas passam pela validação juntas — as duas consultam, as duas não acham ninguém — mas só uma vence o índice.

No 19, a regra: **nunca edite migration que já rodou em produção**. Crie outra.

01:29 · Senha (22–27)

O bloco mais importante da aula inteira.

- **21** — comece pelo susto. Cite o vazamento que você separou.
- **22** — hash não é criptografia. Criptografia tem volta; hash não.
- **23** — por que bcrypt e não SHA-256: SHA é rápido, **e é esse o problema**. GPU testa bilhões por segundo. bcrypt é lento de propósito, com custo ajustável.
- **24** — desmonte o hash na tela: versão, custo, salt, digest.

Demonstre no console — é o momento em que a ficha cai:

```
BCrypt::Password.create("senha123")
BCrypt::Password.create("senha123")    # rode DE NOVO
```

São diferentes. É o salt. Pergunte por que, antes de responder.

01:57 · Validações (29–32)

O 29 (normalizar antes de validar) fecha com o 18: se você não normaliza, o índice único não serve para nada, porque o banco acha que `Ana@UFOP.br` e `ana@ufop.br` são valores diferentes.

02:46 · Associações (34–40)

Bloco novo, e é o que eles mais vão usar no primeiro projeto de verdade.

A regra que resume tudo (slide 32): **a chave estrangeira fica no lado "muitos"**. Quem carrega a coluna usa `belongs_to`; quem é apontado usa `has_many`.

No console:

```
ana = User.first
ana.login_events.create!(client: "mobile", occurred_at: Time.current)
ana.login_events.count
ana.login_events.recentes.first.user.name    # e volta
```

Demonstre o N+1 de verdade (36) — com o log na tela:

```
User.all.each { |u| puts u.login_events.count }    # conte as consultas
User.includes(:login_events).each { |u| puts u.login_events.count }
```

A diferença no log é o argumento. Nenhum slide convence tanto.

03:21 · Concerns (42–45)

A ponte com a Aula 1: *"lembram do módulo que se inclui numa classe? Isto aqui é ele, com nome de Rails."*

O slide 40 tem as três decisões — a que mais rende é a terceira: `email_verified_at` é data, não booleano, porque um dia alguém vai abrir chamado perguntando *quando* a conta foi ativada.

03:55 · Console e testes (47–51)

O slide 51 (`authenticate_by_email`) merece atenção: o ataque de temporização. Se a resposta volta em 1ms quando o e-mail não existe e em 100ms quando existe, dá para descobrir quem tem conta cronometrando. Guarde — volta na Aula 3.

Conduzindo as oito práticas

Cada prática tem, na apostila, a lista de comandos, uma conferência e uma **tabela de erros**. Mande abrir a apostila na prática correspondente — não dite os comandos.

#	Slide	O que cobrar em voz alta
1	10	O <code>docker compose ps</code> tem que dizer healthy , não <code>starting</code>
2	21	Uma migration por vez, e <code>db:migrate</code> entre elas. A conferência é o 17
3	28	Rodar o <code>create</code> duas vezes e ver dar diferente. Isso é o <i>salt</i>
4 e 5	33	Quando <code>valid?</code> der <code>false</code> , o reflexo é <code>p u.errors.full_messages</code>
6	41	O avançado (apagar o usuário e ver os eventos sumirem) vale fazer ao vivo
7	46	O banco guarda o digest do código, não o código
8	52	<code>users(:ana)</code> , nunca <code>User.first</code> , dentro de teste

Os pontos onde a turma trava, em ordem de frequência:

- esqueceram `db:migrate` depois de criar a migration;
- escreveram `has_secure_password` sem adicionar a gem `bcrypt` no Gemfile;
- `matricula` com máscara (`20.112-34`) falhando na validação — é exatamente o motivo de `normalize_matricula` existir;
- `PG::DuplicateTable` de quem rodou a migration duas vezes: `db:drop db:create db:migrate`;
- teste que passa sozinho e falha em conjunto: é `User.first` dentro do teste.

Quando uma migration falhar, a resposta está na **linha seguinte** da mensagem, não na primeira. Ensine isso uma vez e economize meia hora.

Perguntas que vão aparecer

Pergunta	Resposta curta
"Por que não SQLite? É mais simples."	Uma escrita por vez no banco inteiro. Num servidor com várias requisições, trava.
"Dá para descriptografar o <code>password_digest</code> ?"	Não. Não é criptografia, é hash: caminho só de ida. Nem você consegue ver a senha do usuário — e isso é a intenção.
"Por que o bcrypt é lento? Isso não é ruim?"	É lento de propósito . 100ms para você é nada; para quem tenta bilhões de senhas, é o que inviabiliza.
"Se eu já valido no model, para que o índice único?"	Concorrência. Duas requisições ao mesmo tempo passam pelas duas validações e só uma vence o índice.
"Por que não <code>t.boolean :email_verified?</code> "	Porque booleano responde "sim" e data responde "sim, dia 12 às 14h". Um dia alguém vai perguntar quando.
"Posso apagar uma migration antiga e refazer?"	Na sua máquina, sim. Depois que rodou em produção, nunca — crie outra.
" <code>dependent: :delete_all</code> ou <code>:destroy</code> ?"	<code>delete_all</code> é um <code>DELETE</code> só, direto no banco, e não roda callback. <code>:destroy</code> carrega cada registro e roda os callbacks. Aqui não há callback, então <code>delete_all</code> é mais rápido.

Dever de casa

1. Terminar as práticas que não couberam.
2. Bônus: escrever o N+1 de propósito, contar as consultas no log e consertar com `includes`.
3. Ler a seção "Uma sutileza de segurança" da apostila — ela é o começo da próxima aula.

Roteiro da Aula 3: as rotas de autenticação

Deck: `slides/build/aula-03-autenticacao.pptx` (66 slides, sendo 8 de prática) **Apostila da turma:** `03-autenticacao.md` · **Checkpoint:** `aula-03` **Antes de tudo:** `guia-do-instrutor.md` — conduzir a sala, não o conteúdo

Esta é a aula que não cabe. São 1396 linhas de código em 37 arquivos. Digitando, dá 6h30. Leia a seção seguinte antes de qualquer outra coisa.

A decisão que você precisa tomar antes

Você tem duas formas de conduzir esta aula:

Opção A: código pronto, leitura guiada (recomendada)

No começo da aula, todo mundo copia o código do gabarito para **o próprio projeto**:

```
cd ~/capacidade-gabarito && git checkout aula-03
rsync -a app config db test Gemfile Gemfile.lock ~/automic_auth_api/
cd ~/automic_auth_api && bundle install && bin/rails db:migrate && bin/rails test
```

O `Gemfile` vai junto porque a Aula 3 acrescenta `jwt`, `rack-cors` e `letter_opener` — sem ele a aplicação nem sobe. Os 71 testes verdes são a prova de que a cópia deu certo. **Só depois disso** você percorre os arquivos projetados, explicando decisão por decisão, com eles acompanhando no editor.

A prática vira **modificar** o que já existe, que é o que se faz num emprego de verdade. E o código fica no repositório deles, que é de onde a Aula 4 vai fazer o deploy.

Cabe em ~2h50. É o que este roteiro assume.

Opção B: digitar junto

Só funciona se você tiver 6h ou dividir em dois encontros. Se for por aqui, corte para **três rotas**: cadastro, login e logout. Recuperação de senha vira leitura da apostila.

Antes de começar

- [] O `rsync` do gabarito testado num projeto limpo, com `bin/rails test` verde depois.

- [] Servidor rodando e o Insomnia com as cinco requisições já montadas — você vai fazer o fluxo completo ao vivo duas vezes.
- [] jwt.io aberto numa aba.
- [] Um token válido copiado, pronto para colar no jwt.io.
- [] `tmp/letter_opener/` limpo, para o e-mail da demo ser o primeiro da lista.
- [] Editor com `app/services/` e `app/controllers/api/v1/` já abertos na barra lateral.

Cronograma (Opção A)

As oito práticas são o esqueleto do dia, e nesta aula elas são a aula: o código vem pronto, e o que eles fazem é operar o sistema, quebrar de propósito e entender por quê.

Relógio	Slides	Bloco	Min
00:00	1–5	Abertura, de onde paramos e o combinado de hoje	10
00:10	6–15	Arquitetura: service, serializer, erro, filtro, middleware	26
00:36	16	Prática 1 — traga o código e leia	20
00:56	17–19	Cadastro e enumeração de usuários	10
01:06	20	Prática 2 — cadastro no terminal	20
01:26	—	Intervalo	10
01:36	21–24	Mailer, letter_opener, <code>deliver_now</code>	12
01:48	25	Prática 3 — o e-mail e a confirmação	15
02:03	26–41	Sessão, JWT, login, cookie, XSS, CSRF, 401×403	42
02:45	42	Prática 4 — login, e o token na mão	20
03:05	43–49	Logout: denylist e <code>token_version</code>	22
03:27	50	Prática 5 — logout que revoga de verdade	15
03:42	51–56	Recuperação de senha e a transação	16
03:58	57	Prática 6 — recuperação de senha	20
04:18	58–62	Testes, ferramentas, CORS	14
04:32	63–64	Práticas 7 e 8 — qualidade, e quebre a assinatura	30
05:02	65–66	Recapitulação e fim	4

Dá 5h06 com tudo. É a aula com mais conteúdo conceitual das quatro, e agora com prática de verdade em cada bloco. Corte **nesta ordem**:

#	O que cortar	Ganho
1	Prática 8 (quebrar a assinatura) vira dever de casa, com a apostila	-15
2	Slides 37-38 (XSS e CSRF) — o material fica na apostila	-8
3	Slides 61-62 (ferramentas e CORS) — mostre rodando e siga	-8
4	Slides 21-24 (mailer): mostre só o e-mail abrindo no navegador	-8
5	Prática 1: mande fazer o <code>rsync</code> de casa , na véspera; em aula, só a leitura dos 5 arquivos	-8
6	Prática 2: você faz o cadastro projetado, e eles só montam a coleção do Insomnia	-10
7	Prática 6: faça o segundo <code>confere</code> (enumeração) projetado, e o resto de casa	-10

Cortando de 1 a 4, fecha em **4h07**. Cortando os sete, **3h19**.

Nunca corte as Práticas 4 e 5. O token na mão e o logout que revoga são o ponto alto da aula, e são o que diferencia esta capacitação de um tutorial de YouTube.

Bloco a bloco

00:00 · Abertura e o combinado (1-5)

O slide 3 é o mapa da aula inteira. Deixe ele na tela enquanto fala a regra:

Controller recebe requisição e devolve resposta. Regra de negócio não mora nele.

00:10 · Arquitetura (6-15)

Abra `app/services/users/register.rb` projetado e conte as seis coisas que ele faz. Depois pergunte: "quanto disso vocês conseguiriam testar sem subir uma requisição HTTP inteira?"

- **7** — serializer. Faça a demonstração do susto: "o que acontece se eu escrever `render json: user`?"
Mostre no console:

```
ruby User.first.as_json.keys
```

Está lá o `password_digest` e os digests dos códigos. **Uma linha, e vazou.**

- **10-12** — `before_action` e middleware. O slide 12 é o mapa completo. Se der, rode:

```
bash bin/rails middleware
```

- **13** — strong parameters. A frase: "sem isso, alguém manda `role: admin` no cadastro e vira admin. Isso tem nome: mass assignment, e já derrubou sistema grande."

00:56 · Cadastro (17–19)

O slide 19 é o primeiro dos quatro momentos de "enumeração" da aula. Marque isso: você vai voltar nele três vezes, e no fim eles devem conseguir prever a decisão sozinhos.

Pergunta para a turma antes de virar o slide: "o e-mail já existe. O que a API deve responder?" Alguém vai dizer "este e-mail já está cadastrado". Aí você mostra por que não.

01:36 · E-mail (21–24)

Faça o cadastro ao vivo no Insomnia e **mostre o e-mail abrindo no navegador** pelo `letter_opener`. É um daqueles momentos em que a turma acorda.

No slide 24, a decisão contra o livro-texto: `deliver_now` porque, numa fila, o código de 6 dígitos ficaria gravado **em texto** na tabela de jobs.

02:03 · JWT (26–33)

O bloco conceitual mais denso. Comece pelo problema (27): "lembra que HTTP não tem memória? Então como o servidor sabe que vocês já entraram?"

Faça a demo do jwt.io. Cole o token que você preparou e mostre o payload legível na tela.

"Está assinado. Não está criptografado. Qualquer um lê. Então nunca ponham aqui nada que não possa ser lido."

No 33, o `true` do `JWT.decode`. Diga que já foi CVE em várias bibliotecas, e que eles vão brincar com isso na prática.

02:24 · Login (34–41)

- **35** — os dois transportes. O `client` no corpo existe por isso.
- **36–37** — o que é cookie, e as três flags.
- **38–39** — XSS e CSRF. Se o tempo apertar, é aqui que você corta.
- **41** — o segundo "enumeração": mesma mensagem para e-mail inexistente e senha errada. E o `DUMMY_PASSWORD_DIGEST` da Aula 2 volta, fechando o buraco pelo lado do tempo.

03:05 · Logout (43–49)

O ponto alto. Conduza como um problema, não como uma solução.

1. Pergunte: "o token vale 24h e o servidor não guarda sessão. O que 'sair' significa?"
2. Deixe a turma propor. Alguém vai dizer "apaga no cliente". Aceite e pergunte: "e se alguém já copiou o token?"
3. Aí você apresenta a denylist (46).
4. O slide 47 é a sacada: o TTL de cada entrada é o que faltava para o token expirar, então **a lista se limpa sozinha**.

5. E o 48 mostra a outra ferramenta: `token_version` derruba tudo de uma vez.

Demonstre ao vivo. Vale mais que os seis slides:

```
TOKEN=... # pegue do login
curl $API/me -H "Authorization: Bearer $TOKEN" # 200
curl -X DELETE $API/sessions -H "Authorization: Bearer $TOKEN"
curl -i $API/me -H "Authorization: Bearer $TOKEN" # 401
```

03:42 · Recuperação de senha (51–56)

Terceiro e quarto "enumeração" (53). A essa altura, **pergunte antes**: *"o e-mail não existe. O que respondemos?"* Eles devem acertar sozinhos.

O slide 55 amarra com a Aula 2: a transação. E o 56 tem o ponto do fluxo inteiro —

`invalidate_sessions!`: *"se alguém invadiu a conta, trocar a senha tem que expulsar o invasor. Sem essa linha, ele continua logado."*

Demonstre: faça o reset e mostre o token antigo virando 401.

04:18 · Testes, ferramentas e CORS (58–62)

Rápido, e é o que amarra com a Aula 4.

- **59** — o teste que resume a aula: login → logout → `/me` dá 401. Projete-o.
- **60** — a pegadinha de teste: `assert_response :unauthorized`, e não `assert_equal 401`.
- **61** — RuboCop, Brakeman, bundler-audit. Diga que os três viram o **portão** do CI na Aula 4.
- **62** — CORS. A frase: *"o navegador é quem bloqueia, não o servidor. Por isso `curl` funciona e a página não."*

Conduzindo as oito práticas

Nesta aula a prática **é** a aula: o código vem pronto, e o trabalho é operar, quebrar e entender. Cada uma tem, na apostila, os comandos, a conferência e uma tabela de erros.

#	Slide	O que cobrar em voz alta
1	16	71 testes verdes antes de seguir. Sem isso, nada depois funciona
2	20	O <code>Content-Type</code> esquecido é o erro nº 1. E a mensagem do e-mail repetido: entrega quem tem conta?
3	25	Login antes de confirmar dá 403 , não 401. A senha estava certa
4	42	O servidor não guardou sessão nenhuma. Ele leu o token
5	50	O <code>401</code> depois do logout com um token ainda válido. É a prática que justifica a aula
6	57	O e-mail que não existe responde igual . Faça lado a lado, na tela
7	63	Quebrar a denylist e ver qual teste acusa
8	64	Desfazer o <code>false</code> do <code>JWT.decode</code> . Ninguém sai da sala com essa linha

Dois cuidados de condução:

Na Prática 4, se o `$TOKEN` sair vazio para alguém, mande rodar o `curl -i` sozinho e olhar se o cabeçalho `Authorization` está lá. É quase sempre aspas quebradas no shell — e a saída é o Insomnia.

Na Prática 8, cobre o desfazer em voz alta:

```
git checkout app/services/auth/decode_token.rb
bin/rails test
```

É uma falha de autenticação real. Não deixe ninguém ir embora com ela no disco.

Perguntas que vão aparecer

Pergunta	Resposta curta
"Se o payload é público, não é inseguro?"	Não, desde que você não coloque segredo lá. O token prova quem você é; ele não precisa esconder isso de você.
"Por que não usar sessão como todo mundo?"	Porque o cliente principal é app nativo, que não tem cookie. E porque assim qualquer servidor valida sozinho, sem sessão compartilhada.
"Então JWT é pior que sessão?"	É diferente. Sessão facilita o logout e complica o escalar; token faz o contrário. Nós resolvemos o logout com a denylist.
"A denylist não é uma sessão disfarçada?"	Um pouco — mas só guarda os tokens revogados, não todos, e expira sozinha. É bem mais barata.
"Por que 15 minutos no código, e não 1 hora?"	Janela curta reduz o tempo em que um código interceptado serve. 15 min é o suficiente para checar e-mail sem correr.
"E se o e-mail do usuário estiver errado no cadastro?"	Ele nunca confirma, nunca loga, e a conta fica órfã. É por isso que existe o <code>resend</code> .
"Posso usar Devise em vez de escrever tudo isso?"	Pode, e em projeto de verdade normalmente é o que se faz. Mas você não entenderia nada do que ele faz — e é isso que estamos aprendendo.
"Por que <code>unprocessable_content</code> e não <code>unprocessable_entity</code> ?"	Mesmo código 422. O nome foi renomeado na especificação, e o Rails 8 seguiu.

Dever de casa

1. Fazer os dois bônus.
2. Ler `app/services/users/complete_password_reset.rb` inteiro e explicar, por escrito, por que a política de senha é validada **antes** de consumir o código.
3. Rodar `bin/brakeman` e ler o que ele procura.

Roteiro da Aula 4: servidor, Docker, Kamal e deploy

Deck: `slides/build/aula-04-servidor-docker-kamal-e-deploy.pptx` (62 slides, sendo 7 de prática)
Antes de tudo: `guia-do-instrutor.md`, conduzir a sala, não o conteúdo **Apostila da turma:** `04-deploy.md` · **Checkpoint:** `aula-04` **Apêndice:** `apendice-azure.md`, o mesmo percurso na nuvem, para casa

Esta era a aula mais arriscada das quatro, porque o gargalo não era digitar: era clicar em portal, e portal falha. **Agora o servidor é a própria máquina de cada aluno.** Não há VM para criar, nem rede para configurar, nem nuvem para esperar. Sobrou o que realmente ensina.

O que preparar com antecedência

#	O que	Quando
1	Cobrar de todo mundo a saída de <code>ssh "\$USER"@127.0.0.1 'echo ok'</code>	uma semana antes
2	Um SMTP (Resend ou outro) com domínio verificado, e uma key para a turma	véspera
3	Fazer o percurso inteiro sozinho, num usuário limpo, do <code>apt install openssh-server</code> ao <code>curl --cacert</code>	véspera

O item 1 é o que tira o risco do dia

O Kamal precisa de duas coisas na máquina: **SSH que aceite chave** e **Docker rodando**. O Docker eles têm desde a Aula 1. O SSH é o que pode faltar, e descobrir isso em aula custa vinte minutos.

Peça no grupo, uma semana antes:

```
sudo apt install -y openssh-server && sudo service ssh start    # WSL2, Ubuntu, Debian
ssh "$USER"@127.0.0.1 'echo FUNCIONOU'                        # print disto
```

No macOS não se instala nada: **Ajustes do Sistema** → **Geral** → **Compartilhamento** → **Sessão remota**.

O tropeço que vai aparecer: no WSL2 o `sshd` não sobe sozinho a cada boot do Windows. Quem reiniciou entre o teste e a aula vai chegar com `Connection refused`. A saída é uma linha (`sudo service ssh start`), e vale você avisar de véspera para ninguém perder tempo com isso.

Antes de começar

- [] Dois terminais grandes. Hoje **os dois são a mesma máquina**, e vale dizer isso: um é onde você edita e roda o Kamal, o outro é onde você olha `docker ps` e logs.
- [] Um navegador aberto, para o momento do aviso de certificado.
- [] `docs/troubleshooting.md` aberto numa aba.
- [] Portal da Azure logado numa aba anônima, para a demo do fim (não vaze recursos do SEEM na projeção).
- [] O PAT do ghcr.io: avise no grupo, na véspera, para já virem com ele criado.
- [] O seu próprio percurso refeito na véspera, com o `~/ssh/capacita` apagado, para você fazer do zero junto com eles.

Cronograma

As sete práticas são o esqueleto do dia. Estão em negrito, e nesta aula elas são quase tudo: o conteúdo existe para explicar o que a mão está fazendo.

Relógio	Slides	Bloco	Min
00:00	1–3	Abertura, e de onde paramos	6
00:06	4–10	O problema, as opções, e por que a máquina é a sua	18
00:24	11–18	Chaves, SSH, e autorizar a si mesmo	20
00:44	19	Prática 1: a sua máquina vira o servidor	20
01:04	20–25	Docker: imagem, Dockerfile, registry, arquitetura	16
01:20	26	Prática 2: arquivos de deploy e o token	15
01:35	—	Intervalo	10
01:45	27–35	Kamal: <code>deploy.yml</code> , accessory, segredos	22
02:07	36	Prática 3: o <code>.env</code>	15
02:22	37–44	TLS, certificado, CA, proxy reverso	22
02:44	45	Prática 4: certificado e <code>/etc/hosts</code>	15
02:59	46–50	CI, o primeiro deploy, operar, backup	14
03:13	51–52	Práticas 5 e 6: o deploy, e o certificado com os olhos	35
03:48	53–56	Num servidor de verdade: demo	15
04:03	57	Prática 7: o sistema inteiro, e o backup	20
04:23	58–62	O que vocês fizeram, e agora, fim	8

Dá 4h31. Caiu ~45 minutos em relação à versão com VM, e o que saiu foi exatamente o que não ensinava back-end: criar máquina, esperar download de imagem e configurar rede.

Corte **nesta ordem**:

#	O que cortar	Ganho
1	Prática 7 vira dever de casa (fluxo da Aula 3 em produção, e o backup)	-20
2	A demo de servidor de verdade (53-56) encolhe para 6 min: só os slides 54 e 55	-9
3	Slides 22-23 (Dockerfile e as duas decisões) ficam na apostila	-8
4	Slides 34-35 (credentials × ENV, três camadas) ficam na apostila	-8
5	Prática 4: você gera um certificado e distribui; eles só fazem o <code>/etc/hosts</code>	-8
6	Prática 2: mande fazer o <code>rsync</code> e o PAT de casa , na véspera	-12

Cortando de 1 a 3, fecha em **3h54**. Cortando os seis, **3h18**.

Nunca corte as Práticas 5 e 6. Ver a própria API no ar e entender o aviso de certificado são os dois momentos que a turma leva embora.

Onde o tempo ainda pode escapar

Risco	Sinal	O que fazer
<code>sshd</code> parado	<code>Connection refused</code> na Prática 1	<code>sudo service ssh start</code> . É uma linha, e é o erro mais comum do dia
Permissão de chave	<code>Permission denied (publickey)</code>	<code>chmod 700 ~/.ssh</code> e <code>chmod 600</code> na chave e no <code>authorized_keys</code>
Build da imagem	<code>kamal setup</code> demorando	Normal na primeira vez. Use o tempo para o slide de backup e para perguntas
Arquitetura errada	<code>exec format error</code> no container	<code>uname -m</code> . Quem tem Mac com chip M precisa de <code>arm64</code>

Plano B, decidido de véspera

B1: cortar o TLS. Se às 02:22 a maioria ainda não deployou, tire `proxy.ssl` do `deploy.yml`, suba em HTTP puro, e faça o bloco de TLS todo projetado, no seu. **Devolve ~20 min.** Eles refazem em casa com a apostila.

B2: virar demo a partir do intervalo. Se às 01:35 não houver pelo menos metade da turma com o `SSH_OK` na tela, pare de esperar e projete o percurso do começo ao fim.

Bloco a bloco

00:06 · O problema e as opções (4–10)

Comece pelo slide 5, que é a pergunta honesta: `bin/rails server` **funciona enquanto o terminal está aberto — e daí?** As cinco necessidades listadas ali são a aula inteira, e nenhuma depende de onde a máquina está.

O slide 9 é o que dá credibilidade: admita o custo. Uma máquina só é ponto único de falha, volume não é backup, o Postgres não é gerenciado.

O **10** é o slide que define o dia:

"O Kamal não sabe onde a máquina está. Ele conecta por SSH e roda `docker` do outro lado. Se o endereço for 127.0.0.1, ele conecta na de vocês."

Diga com todas as letras que não é faz de conta. Alguém vai desconfiar, e com razão: é a mesma conexão SSH, o mesmo `deploy.yml`, os mesmos comandos. O que um servidor alugado acrescenta são duas linhas do `.env` e o trabalho de provisionar a máquina, que está no apêndice.

00:24 · Chaves e SSH (11–18)

- **12** — o Kamal precisa de duas coisas, e uma delas eles já têm. Isso desarma a sensação de que hoje é tudo novo.
- **13–14** — chave pública e privada. É o conceito que também explica o JWT da Aula 3 e o TLS que vem daqui a pouco. **Não pule**, mesmo com a turma ansiosa para digitar.
- **15** — instalar o `ssh`. **Avise aqui, antes de alguém apanhar:** no WSL2 ele não sobe sozinho no boot.
- **16** — autorizar a si mesmo. Vale a pausa: *"isto é literalmente o que vocês fariam num servidor alugado. A mesma chave, o mesmo arquivo, o mesmo comando. Muda o endereço."*
- **17** — o `chmod 600`. Diga que o SSH **recusa** chave com permissão frouxa, e que é a primeira coisa a conferir no `Permission denied (publickey)`.

Ponto de decisão às 01:04: se menos da metade tiver o `SSH_OK`, resolva junto antes de seguir. Sem isso, nada depois acontece.

01:04 · Docker (20–25)

Conceitual e rápido. Imagem é receita, container é bolo.

No **23**, as duas decisões: multi-stage (compilador não vai para produção) e usuário não-root (*"é uma linha que muda o tamanho do estrago"*).

O **24** explica por que a imagem sobe para o ghcr.io se quem builda e quem roda são a mesma máquina. **Alguém vai perguntar, e a pergunta é boa.** A resposta: é o que aconteceria com um servidor do outro lado do mundo, e é o que faz o mesmo `deploy.yml` servir nos dois casos sem mudar uma linha.

O **25** é operacional: PAT com `write:packages`, e a arquitetura. **Circule pela sala aqui:** quem tem Mac com chip M precisa de `arm64`, e quem errar isso só descobre no meio do `kamal setup`, com uma mensagem que não diz "arquitetura".

01:45 · Kamal (27–35)

Abra o `config/deploy.yml` projetado e percorra junto com os slides.

O slide 30 (`clear` × `secret`) tem o argumento concreto: `JWT_SECRET` no `clear` apareceria em `docker inspect` e no histórico do shell. Se der, mostre um `docker inspect` de verdade.

O **34** (três camadas de segredo) é o slide que amarra tudo. Deixe na tela enquanto explica, e diga a linha que conecta com o apêndice: *"quando isso virar um pipeline, só a primeira camada muda."*

No **35**, pare no `.env`: `git status` **não pode mostrar esse arquivo**. Mande todo mundo rodar, ali, na frente de você.

02:22 · TLS (37–44)

O melhor bloco da aula.

- **38–41** — HTTPS é HTTP num túnel; handshake em três passos; certificado e cadeia de confiança.
- **41** é a virada: *"para uma CA pública assinar, você tem que provar que o domínio é seu. E vocês não têm domínio nenhum."* É daí que sai o autoassinado, como consequência e não como gambiarra.
- **43** — `.test` e `/etc/hosts`. Vale a curiosidade: `.test` é reservado por RFC exatamente para isso, e ninguém consegue registrar.
- **44** — **o slide da aula**. Não passe rápido.

03:13 · O deploy, e o certificado (51–52)

```
source .env && bundle exec kamal config
```

Valida sem tocar em nada. **Faça isso antes:** pega variável faltando de graça.

Depois `kamal setup`. Use a espera do build para o slide de backup e para perguntas.

E então, o momento da aula. Diga o que está na frente deles, porque nem todo mundo percebe sozinho:

"O código de vocês está rodando dentro de um container, em modo produção, atrás de um proxy que termina TLS, com um Postgres que tem volume. Vocês não estão mais rodando `bin/rails server`."

Na Prática 6, os três `curl` respondem, um a um, o que TLS é e o que CA é:

```
curl https://seunome.test/...      # falha: self signed certificate
curl -k https://seunome.test/...    # funciona, sem conferir nada
curl --cacert tls/capacita-cert.pem ... # funciona, CONFERINDO
```

A frase que fixa: "o `-k` desliga a conferência. O `--cacert` não desliga nada, ele diz em quem confiar. Uma CA é isso e só isso: alguém em quem o seu sistema já decidiu confiar, de fábrica."

Depois abra no navegador e leia o aviso vermelho junto com eles. **É o mesmo fato, dito para um humano.**

03:48 · Num servidor de verdade, demo (53–56)

Quinze minutos, projetado, sem ninguém acompanhando no teclado. Diga isso antes de começar, ou metade da turma vai tentar criar conta na Azure e perder o fim da aula.

O slide 54 é o que importa: no `.env`, **duas linhas**. O resto do trabalho é provisionar a máquina, e é isso que você percorre no portal: VM → usuário → `apt upgrade` → Docker → firewall → `sshd` endurecido → DNS → certificado → OIDC.

Não detalhe nenhum. O que você quer é que reconheçam as peças e saibam que o passo a passo existe.

Feche apontando o apêndice: `docs/apendice-azure.md`, e a **Azure for Students dá US\$100 sem cartão**.

04:23 · Prática 7 e fecho (57–62)

O slide 59 (`O QUE O SEEM-BACKEND TEM A MAIS`) é o convite: *"nada disso é difícil depois do que vocês viram hoje. O código está lá, e agora vocês conseguem ler."*

E então **reserve cinco minutos de verdade para os slides 60 e 61**. Não corra.

O **60** (`O QUE VOCÊS FIZERAM`) é a lista do que construíram, e existe porque quem fez raramente percebe o tamanho: o esforço esconde a conquista. Leia devagar, olhando para eles.

O **61** (`E AGORA?`) deixa o canal aberto. Diga em voz alta que dúvida daqui a três semanas ainda é dúvida: a pergunta que chega depois costuma ser a mais importante da capacitação inteira, porque é a primeira que veio de um problema real deles.

Perguntas que vão aparecer

Pergunta	Resposta curta
"Isso não é 'de verdade', é só na minha máquina."	É de verdade: é o mesmo Docker, o mesmo Kamal, o mesmo <code>deploy.yml</code> , a mesma conexão SSH. O que falta é o IP ser público, e são duas linhas no <code>.env</code> .
"Então por que não usar a nuvem direto?"	Porque metade da aula viraria espera de portal e de cota, e nada disso é back-end. O apêndice tem o caminho, e vocês já vão saber operar quando chegarem lá.
"Por que a imagem vai para o GitHub se roda aqui mesmo?"	Porque é o que aconteceria com um servidor remoto, e é o que faz o mesmo arquivo servir nos dois casos. Também é o que te dá versão e rollback de graça.
"Por que não Heroku/Render/Vercel? É mais fácil."	É mesmo, e para muita coisa é a escolha certa. Aqui o objetivo é entender o que essas plataformas fazem por vocês, e o custo delas cresce rápido.
"Por que não rodar só com Docker Compose?"	Compose sobe container. Não resolve registry, build versionado, zero-downtime, rollback nem gestão de segredo. É o que o Kamal acrescenta.
"Meu navegador diz que o site não é seguro."	E está certo: o certificado é seu, assinado por você. É a Prática 6, e é a diferença entre criptografia e confiança.
"E se eu quiser desligar isso depois?"	<code>kamal app stop</code> e <code>kamal accessory stop db</code> . Para apagar de vez, <code>kamal remove</code> .
"Posso usar isso num projeto meu?"	Pode, e é a intenção. Troque o nome do serviço, o host e os segredos. O resto é igual.

Depois da capacitação

Mande no grupo:

1. `kamal app stop` e `kamal accessory stop db` para quem não quiser os containers rodando.
2. `apendice-azure.md`, com o link da Azure for Students, para quem quiser o IP público e o deploy automático.
3. O link do `seem-backend`, com o convite do slide 59.
4. `docs/troubleshooting.md`, que é o que eles vão abrir quando algo quebrar sem você por perto.